

FARO — Security, protocols and data handling

Book 1 of 3 · For the institution, the IT team and the jury

Version: September 2026 · SHIFT 2026 Challenge · Gamification

What this book is for

A university that connects a tool to its Canvas is lending something valuable: access to its students' academic data. This book explains, plainly, what FARO does with that access, which protocols it uses, why those protocols are secure, and which guarantees can be **verified** instead of believed.

Every claim in this book carries one of three marks:

- **[code]** — it is written in the repository and the file is cited.
- **[test]** — an automated test verifies it (`server/test/e2e.ts`).
- **[pending]** — it is the production design and is not implemented yet.

Anything without a mark is context.

Contents

1. Executive summary
2. Trust architecture: where every piece of data lives
3. Protocols: why they are secure
4. The Canvas token: life cycle
5. Data minimisation and retention
6. Threat model and controls
7. Compliance: LFPDPPP and institutional policies
8. Frequently asked questions for the university
9. Pre-pilot checklist

The five guarantees, on one page

Guarantee	How it is met	Mark
FARO only reads. It cannot modify grades, submissions, due dates or content.	Every call is a <code>GET</code> . The backend refuses any other method before touching the network. The LTI write scopes (<code>score</code> , <code>lineitem</code>) are explicitly refused.	[code] [test]
The credential is never in the browser.	The token lives in <code>server/.env</code> and in the backend's memory. The extension has no field to hold one and no network permission for <code>instructure.com</code> .	[code] [test]
Only six Canvas paths, and nothing else.	An allow list in the backend (<code>server/src/allowlist.ts</code>) mirrored in the extension. A path outside the list is refused locally, with no call to Canvas.	[code] [test]
The AI mentor sees eleven fields and no personal data.	The context is built in a single file (<code>src/lib/mentorContext.ts</code>) that has no access to name, email, photo or grades.	[code]
A risk score is never shown to the student.	Friction states exist only to choose the intervention; the interface uses supportive language.	[code]

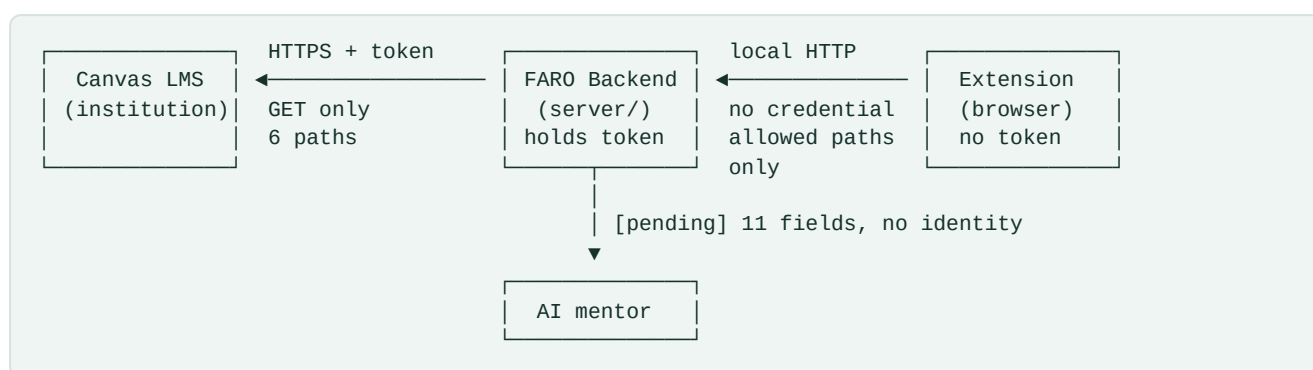
1. Executive summary

What FARO is, in terms of data

FARO is a persistence companion for adult learners studying online. It reads the course structure and the student's progress from Canvas, detects when someone is drifting away, and offers them a route back. It does not teach the course, it does not grade, it does not replace Canvas.

From the institution's point of view, FARO is **a read-only reader** of a small, well-defined slice of Canvas, with an artificial-intelligence layer that receives a minimal, anonymised context.

The three pieces and who talks to whom



- **Canvas** remains the academic source of truth. FARO does not write to it.
- **The backend** is the only process that knows the Canvas credential. It is a Node.js server with no third-party dependencies (`server/`), which reduces supply-chain risk in the token-holding process to zero.
- **The extension** is interface only. It has no token, no network permission towards Canvas, and can only ask the backend for paths the backend has already decided to allow.
- **The AI mentor** receives a hand-built context of eleven fields, none of which identifies the person.

The security posture, in one sentence

Reduce what FARO *can* do until it matches what FARO *needs* to do, and make that reduction verifiable by an administrator without trusting us.

Current state, honestly

Component	State
Backend with token, allow list, restricted CORS, rate limit, no token leakage in logs	Implemented and tested (20 automated checks)

Component	State
Extension with no credentials, <code>http</code> mode against the backend	Implemented
Mentor data contract (11 fields)	Implemented
Tests against a real Canvas (free Instructure account)	Ready to run ; needs a pilot token
Signed LTI 1.3 launch (OIDC + JWT)	Designed and documented , routes present and answering 501; needs the institution to register the tool
Per-student token via OAuth2 (instead of one shared token)	Designed ; depends on the LTI registration
PostgreSQL database with encryption at rest	Pending ; today FARO's state lives in the student's browser
Privacy review with the institution	Pending ; this book is the input

2. Trust architecture: where every piece of data lives

The question that matters most in a security review is not "which protocol do you use?" but "where is each piece of data and who can reach it?". This is the complete answer.

2.1 Trust zones

FARO has four zones. Each piece of data lives in exactly one, and crosses a boundary only for a documented reason.

Zone	What it holds	Who has access	Trust
Z1 · Canvas	The whole academic record	The institution	Highest. FARO neither controls nor alters it.
Z2 · FARO backend	The Canvas token (in memory), the configuration, a short-lived cache of the launch	The backend operator	High. It is the only process with a credential.
Z3 · The student's browser	Purpose, preferences, sessions recorded by FARO, optional profile, a snapshot of the course while the tab is open	The student	Medium. It is their own device.
Z4 · AI service	Eleven context fields per conversation	The model provider	Controlled by a data contract. Never receives identity.

2.2 Data inventory

Each row answers: what it is, where it comes from, where it is stored, who sees it, how long it lasts.

Data that comes from Canvas (Z1 → Z2 → Z3)

Data	Canvas path	Stored in	Retention
Course name, code and end date	<code>GET /api/v1/courses/:id</code>	Browser memory while FARO is open	Discarded on close
Modules, their items and the student's completion state	<code>GET /api/v1/courses/:id/modules?include[]=items</code>	Same	Same

Data	Canvas path	Stored in	Retention
Assignments with the student's submission (date, state, score)	<code>GET /api/v1/courses/:id/assignments?include[]=submission</code>	Same	Same
Page views per hour and participations	<code>GET /api/v1/courses/:id/analytics/users/:uid/activity</code>	Same. Only the timestamps are kept ; the view counts are discarded in the mapping.	Same
The student's active courses	<code>GET /api/v1/courses?enrollment_state=active</code>	Backend, only to pick the course when none is configured	One process
Numeric id of the token owner	<code>GET /api/v1/users/self</code>	Backend, cached. Name and avatar in that response are discarded.	One process

[code] `src/data/canvas/endpoints.ts`, `src/data/canvas/client.ts` (`mapActivity`), `server/src/routes/launch.ts`.

Data FARO creates (Z3)

Data	Why it exists	Stored in	Leaves the browser
Purpose (goal + destination sentence + minutes/day)	Personalise the route and the mentor	<code>chrome.storage.local</code>	Goal, sentence and minutes are part of the mentor context. Nothing else.
Study sessions recorded by FARO	Momentum, achievements, points	<code>chrome.storage.local</code>	No
"Life happened" state	Choose the intervention	Memory	No

Data	Why it exists	Stored in	Leaves the browser
Mentor style, language, theme	Preferences	<code>chrome.storage.local</code>	Style and language are part of the mentor context
Name, photo (downsized), bio	Optional profile in Community	<code>chrome.storage.local</code>	Not to the mentor. In production, to other students in the course if the student enables it.
Conversation with the mentor	The current session	Memory	Not today: the mentor is local. In production, each message will go to the model with the eleven-field context, through the backend

[code] `src/state/storage.ts` , `src/state/store.tsx` , `src/types/index.ts` (comments on `Profile`).

Data that is never collected

This list matters as much as the ones above. FARO does **not** ask for, read or store:

- Canvas or institutional passwords.
- The student's email address.
- A list of classmates by name (the enrollments endpoint is not called; **[test]** the `/enrollments` path is refused).
- Submission contents (texts, files, exam answers).
- Other students' grades.
- Location, the student's IP address (the backend keeps only the client address for rate limiting, in memory, never persisted).
- Browsing history beyond the listed Canvas paths.

2.3 The boundaries and what crosses them

Z1 Canvas → Z2 Backend	backend's token; full response of the 6 paths
Z2 Backend → Z3 Browser	Canvas JSON, already paginated; NEVER the token; NEVER Canvas header
Z3 Browser → Z2 Backend	the requested path (no credential); the backend checks it against the
Z3 Browser → Z2 → Z4 AI	[pending] 11 fields + the message; NEVER name, email, photo, ids, gr

What is not in the diagram does not happen either: the extension does not talk to Canvas (no `host_permissions` for `instructure.com` **[code]** `public/manifest.json`), and today **no data leaves towards an AI service**: the prototype's mentor is local and deterministic (`LocalMentorService`), with no API key. The `HttpMentorService` → `POST /api/mentor` path already exists in the extension for when the backend takes on the model call (chapter 5).

2.4 Why the backend has no dependencies

The process that holds the institutional credential uses only Node.js built-in modules (`node:http` , `node:crypto` , `node:fs`). There is no `node_modules` in production. This is not aesthetic minimalism: every third-party package running inside the token process is code the institution did not write and that could, in a compromised update, read the token from memory. Zero packages is zero surface of that kind.

[code] `server/package.json` (`"dependencies": {}`).

3. Protocols: why they are secure

FARO invents no security mechanism. It uses the same standards Canvas requires of any integration and applies them in the most restrictive way the product allows. This chapter explains each one: the problem it solves, how FARO uses it, and what it guarantees.

3.1 TLS (HTTPS) — the channel

Problem it solves: nobody on the network (a café Wi-Fi, an ISP, a badly configured institutional proxy) can read or alter what travels between FARO and Canvas.

How FARO uses it: the backend refuses to start if `CANVAS_API_URL` does not begin with `https://`. The only exception is `localhost`, and it exists solely for the automated tests against the fake Canvas.

[code] `server/src/env.ts`, `loadEnv`.

Guarantee: the Canvas token only travels encrypted. Canvas, for its part, only accepts the API over HTTPS.

3.2 Bearer token — the API credential

Problem it solves: proving to Canvas who is making the request without sending a password.

How FARO uses it: the backend adds `Authorization: Bearer <token>` to every request to Canvas, which is the method Instructure recommends ("If possible, using the HTTP Authorization header is recommended"). The token never goes in the URL, never in the body, never in a log.

[code] `server/src/canvas.ts`, `canvasGet`. **[test]** "the token was sent to Canvas on every call, and only there" and "log lines carry no query strings".

Guarantee: Instructure describes tokens as "password equivalent". FARO treats them exactly that way: they live in a single process, in memory, and are read from a file that git ignores. Chapter 4 details their life cycle.

Pilot limitation, stated: a manually generated personal token is what Instructure indicates for testing ("For testing your application before you've implemented OAuth, the simplest option is to generate an access token on your user's profile page"). For multiple users, their documentation is blunt: "Applications in use by multiple users MUST use OAuth to obtain tokens". FARO meets that with the LTI + OAuth2 path in section 3.4. **[pending]**

3.3 Path allow list — the scope

Problem it solves: a token with more permissions than needed (a student's personal token can read everything the student sees in Canvas) is used only for what FARO needs.

How FARO uses it: the backend compares every requested path against six regular expressions. `GET` only. Only the parameters `include[]`, `per_page`, `enrollment_state` and `page`, with validated values. Anything else is refused with `403` **before touching the network**. The path is decoded before comparison, so `%2e%2e` cannot escape the pattern.

[code] `server/src/allowlist.ts`. **[test]** "a path outside the allow list is refused locally (no Canvas call)", "an unknown query parameter is refused", "path traversal is refused", "a write is refused even on an allowed path".

Guarantee: even if the extension were modified by someone, it cannot turn the backend into a general proxy to Canvas. And even though the pilot token could read the list of classmates, FARO **cannot ask for it**.

3.4 LTI 1.3 + OpenID Connect + JWT — the institutional launch

This is the production path. It is designed and documented in the code; the routes exist and answer `501` until the institution registers the tool. **[pending]**

Problem it solves: knowing with certainty *which course* and *which person* are opening FARO, without asking anyone to identify themselves and without FARO having to trust what the extension says.

How it works (IMS Security Framework 1.0):

1. Canvas initiates an OpenID Connect *login* towards `GET /lti/login`. FARO stores a short-lived `state` and `nonce` and redirects to Canvas's authorisation endpoint.
2. Canvas answers with `POST /lti/launch` and an `id_token`: a JWT **signed by Canvas** with RS256.
3. FARO downloads Canvas's public keys (JWKS), verifies the signature, the issuer (`iss`), the audience (`aud` = FARO's `client_id`), the expiry, and that the `nonce` is the one issued and has not been used.
4. From the verified claims, FARO reads exactly two things: the course (`https://purl.imsglobal.org/spec/lti/claim/context`) and the person (`sub`).
5. FARO issues its own session (an `HttpOnly`, `Secure`, `SameSite=None` cookie, because it runs inside a Canvas iframe) bound to that course and that person.

[code] `server/src/routes/lti.ts` (full specification in the comments).

Guarantee: the student's identity is asserted cryptographically by Canvas, not by the extension. FARO cannot be tricked into showing someone else's course because it accepts no identity claim that is not signed by the institution.

LTI scopes: FARO requests `contextmembership.readonly` (NRPS, for aggregate presence counts) and `result.readonly` (AGS, read-only results). It **refuses** `score` and `lineitem`, the two scopes that would allow writing to the gradebook. An administrator can verify this on the developer key screen without reading a line of code.

[code] `src/data/canvas/endpoints.ts`, `ltiScopes`.

3.5 OAuth2 with a developer key — the per-student token

[pending] Once the tool is registered, each student obtains their own access token through Canvas's OAuth2 flow, with scopes limited to the six paths (in the `url:GET|/api/v1/...` form Canvas uses on the developer key). Tokens from this flow expire in one hour and are renewed with a *refresh token*, both stored only in the backend. They are revoked with `DELETE /login/oauth2/token`.

Guarantee: the pilot's shared token disappears. Each student sees exactly what Canvas lets them see, and revoking FARO's access for one person is a single-token operation.

[code] the scopes are already written in `src/data/canvas/endpoints.ts`, `requiredScopes`.

3.6 CORS — who may call the backend from a browser

Problem it solves: any web page open in the same browser using the FARO backend as if it were the extension.

How FARO uses it: the backend keeps a list of allowed origins (`FARO_ALLOWED_ORIGINS`). An origin outside the list receives `403` with no data and no CORS headers. The advertised methods are only `GET`, `OPTIONS`.

[code] `server/src/http.ts`, `corsHeaders`. **[test]** "a browser origin outside the list gets 403 and no data", "CORS preflight succeeds for an allowed origin".

3.7 Rate limit — protection against loops

Problem it solves: an extension with a bug that retries in a loop could send thousands of requests to Canvas with the institutional token, and Canvas could suspend that token.

How FARO uses it: a per-client counter over a one-minute window (`FARO_RATE_LIMIT_PER_MINUTE`, 120 by default). Beyond it, `429` with `Retry-After`.

[code] `server/src/http.ts`, `RateLimiter`. **[test]** "rate limit trips after the configured burst".

3.8 Response headers

Every backend response carries `Cache-Control: no-store` (no academic data stays in intermediate caches), `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`.

[code] `server/src/http.ts`, `send`. **[test]** "responses carry no-store and nosniff".

3.9 Summary: which protocol protects what

Threat	Protocol / control	State
Eavesdropping on the network	Mandatory TLS	[code]
Credential theft from the browser	The browser has no credential	[code] [test]
Token used beyond what is needed	Allow list of 6 paths, GET only	[code] [test]
Student impersonation	LTI 1.3 with a JWT signed by Canvas	[pending]
Token shared between people	OAuth2 per student	[pending]
Malicious page using the backend	CORS with an origin list	[code] [test]
Request loop	Rate limit	[code] [test]
Writing to the academic record	No write scopes; <code>score / lineitem</code> refused	[code]

4. The Canvas token: life cycle

The token is the most sensitive piece of data FARO handles. This chapter follows its whole life: where it is born, where it lives, where it travels, where it is **never**, and how it dies.

4.1 Birth

Pilot (today): generated by the owner of the Canvas account, under *Account* → *Settings* → *Approved Integrations* → *New Access Token*. Canvas shows it only once. Two Canvas details work in our favour:

- Institutions can **require an expiry date** for student tokens (Instructure documents a default of 30 days for students and 90 for instructors), and can disable manual token generation altogether.
- The token inherits exactly the permissions of the user who created it: a student's token cannot see more than that student.

Production: issued by Canvas through OAuth2 from a developer key registered by the institution, with limited scopes and a one-hour expiry. **[pending]**

4.2 Residence

The token lives in **two places and only two**:

1. `server/.env`, a file on the machine that runs the backend. It is in `.gitignore` **[code]** and the repository ships `server/.env.example` with the field empty so nobody has to guess the format.
2. The backend process's memory, in the `Env` object, from start-up to shutdown.

4.3 Travel

It leaves the backend in a single direction: towards Canvas, in the `Authorization` header, over HTTPS, and only for allow-listed paths. Before following a pagination link (`Link: rel="next"`), the backend verifies the URL still points at the configured Canvas host; a link pointing to another domain is refused, because following it would send the token elsewhere.

[code] `server/src/canvas.ts`, the `url.startsWith(env.canvasApiUrl + '/')` check.

4.4 Where it never is

Place	Why not	Verification
The extension	<code>CanvasConfig</code> has no token field. The variable does not exist.	<code>src/data/canvas/config.ts</code>

Place	Why not	Verification
The Vite build	Vite only exposes <code>VITE_*</code> variables; the token has no such prefix and is not in the root <code>.env</code> .	The root <code>.env.example</code> says so explicitly
Backend logs	The logger receives method, path without query, status, duration and an id. Never headers.	[test] "the token was sent to Canvas on every call, and only there"
Backend responses	<code>describe(env)</code> is the only representation of the configuration that leaves, and it contains <code>tokenPresent: true/false</code> , not the token.	[test] "health names the Canvas host and never the token"
Error messages	<code>CanvasError</code> stores status and path, never the body of Canvas's response.	<code>server/src/canvas.ts</code>
The git repository	<code>.gitignore</code> excludes <code>.env</code> in any folder.	<code>.gitignore</code>
The AI service	The mentor context has no credential field.	<code>src/lib/mentorContext.ts</code>

4.5 Rotation

Changing the token is editing `server/.env` and restarting the backend. There is no token cache anywhere else that survives a restart. Pilot recommendation: tokens with a 30-day expiry, rotated when they expire.

4.6 Revocation

- **From Canvas:** the token owner deletes it under *Approved Integrations*. From that instant the backend receives `401` and reports it as `canvas_token_rejected` without trying anything else. **[test]** "a wrong token is reported as rejected, not as a generic failure".
- **From the backend:** clear the value in `.env` and restart. The backend refuses to start without a token, which makes a "started without a credential and nobody noticed" state impossible.
- **Production (OAuth2):** `DELETE /login/oauth2/token` revokes the student's token in Canvas. **[pending]**

4.7 If the token leaked

Scenario: someone gains access to `server/.env`.

1. **Blast radius:** reading what that Canvas user can read. No writing: a student's token cannot alter their record, and an instructor's cannot either through FARO, although it could directly against Canvas. That is why the pilot uses **student-account tokens**, never instructor or administrator ones.
2. **Containment:** revoke the token in Canvas (one click). Generate a new one. Restart the backend.

3. **Detection:** Canvas records the use of every token; a usage pattern from an unknown IP is visible to the institution.
4. **Structural prevention:** the LTI + OAuth2 path removes the long-lived `.env` token file and replaces it with one-hour tokens issued per student.

5. Data minimisation and retention

FARO's guiding principle for data is simple: **collect a piece of data not because it is technically possible, but because a specific feature needs it**. This chapter shows how that applies in each layer, and what happens to data over time.

5.1 Minimisation in the Canvas layer

Canvas returns far more than FARO uses. The integration layer discards what it does not need **at the moment of reading it**, not afterwards.

What Canvas sends	What FARO keeps	What is discarded
Course object (dozens of fields)	id, name, code, start and end dates	Everything else: settings, permissions, image, etc.
Modules with items	id, name, position, state, item count, each item's <code>content_id</code> and whether it is completed	Page titles, external URLs, files
Assignments with submission	id, name, due date, points possible, and from the submission: date, state, score	Assignment description, rubric, attachments, instructor comments
Student activity	Timestamps only. The number of views per hour is removed.	View counts, visited URLs
<code>users/self</code>	The numeric id	Name, avatar, email, time zone

[code] `src/data/canvas/raw.ts` declares only the fields that are read; `src/data/canvas/client.ts` does the mapping; `server/src/routes/launch.ts` keeps only `self.data.id`.

Why the activity case matters: "how many pages they viewed at 3 pm" is surveillance data. "When they last came in" is the only thing the friction engine needs. FARO keeps the second and destroys the first.

5.2 Minimisation in the AI mentor

The context the model receives is built in a single function and contains exactly these eleven fields:

```
{
  "course": "Gestión de Proyectos",
  "progress": 44,
  "next_activity": "Caso de estudio: alcance",
  "estimated_time": 25,
  "student_goal": "career_growth",
  "destination": "Liderar mis propios proyectos",
  "available_time": 20,
  "momentum": 62,
  "friction_state": "FRICTION",
  "style": "encouraging",
```

```
"language": "es"
}
```

No name, email, Canvas id, grades, history of other conversations or dropout-risk classification. The friction state is one of five intervention labels (`FLOWING` , `FRICTION` , `POSSIBLE_OVERWHELM` , `DISCONNECTION` , `RECOVERY`), not a score.

The only human-authored information is the **destination sentence**, which the student writes freely at onboarding. The example shown is *"Lead my own projects instead of just executing them"*, steering towards a goal, not personal data. In production this sentence must carry an explicit warning: *"do not include anything that identifies you"*. **[pending]**

[code] `src/lib/mentorContext.ts` , `src/types/index.ts` (`MentorContext` interface).

5.3 Minimisation in Community

Community is the part of FARO with the greatest temptation to collect social data, which is why it has the strictest rules, written into the types:

- **There is no follower field and no rank.** They are not hidden: they are not in the data model, so no function can compute them.
- **Presence is aggregate.** "23 students studying now" is a count. "Andrea has been at it for 37 minutes" never appears, and the `CoursePresence` type only admits numbers.
- **Classmates have no photo.** Only initials and a decorative colour. Only the student themselves can add an image to their avatar, downsized in their browser before being saved.
- **Reactions are shown separately**, never summed into a score.

[code] `src/types/index.ts` , comments on `CommunityAuthor` and `CoursePresence` .

5.4 Retention

Data	Where	How long	How it is deleted
Course snapshot (from Canvas)	Browser memory	While FARO is open	Closing the tab or panel
Canvas request log	Browser memory, 30 entries	While FARO is open	Automatic; never persisted or sent
Purpose, preferences, sessions, profile	<code>chrome.storage.local</code>	Until the student deletes it	<i>Reset</i> button in Progress, or uninstalling the extension
Canvas token	<code>server/.env</code> + backend memory	Until rotation or revocation	Edit <code>.env</code> and restart; revoke in Canvas
Launch cache (course id, user id)	Backend memory	Until the process restarts	Restart

Data	Where	How long	How it is deleted
Backend logs	Standard output	Defined by the operator	Defined by the operator; they contain no personal data or query strings

Production [pending]: once PostgreSQL exists, the proposal is:

- Study sessions, purpose and preferences: while the enrolment is active, plus 90 days to allow "returning" to a paused course; then automatic deletion.
- Mentor conversations: not stored beyond the session unless explicit consent is given to improve the product, in which case they are anonymised.
- Friction events and interventions: aggregated for pilot metrics, with the student id replaced by a pseudonymous id that only the institution can resolve.

5.5 What the student can see and control

- **Profile** → **Canvas** shows, live, every call FARO makes, with its path, status and latency. Transparency is not a document: it is a screen.
- **Profile** → **Canvas** shows the LTI scopes FARO refuses.
- **FARO Mentor** shows, at the bottom, the exact list of what the mentor knows and does not know.
- **Progress** → **Reset** wipes all of FARO's local state.
- Nothing in the profile is mandatory. FARO works fully without a name or photo.

6. Threat model and controls

A threat model answers three questions: what we want to protect, from whom, and what we have done about it. This table is the answer, threat by threat. The *Test* column names the automated check in `server/test/e2e.ts` when one exists.

6.1 Assets

1. **The Canvas token** — grants read access to its owner's record.
2. **The student's academic data** — progress, submissions, dates.
3. **The student's identity** — who the person behind a FARO session is.
4. **The integrity of the record** — nothing FARO does can alter Canvas.
5. **The student's trust** — that FARO does not turn into surveillance.

6.2 Threats and controls

#	Threat	Asset	Control	State	Test
T1	An attacker extracts the credential from the extension's code (extensions are public code)	Token	There is no credential in the extension. <code>CanvasConfig</code> has no field for it.	[code]	—
T2	A modified extension uses the backend to read any Canvas path	Data	Allow list of 6 paths in the backend; local <code>403</code> with no Canvas call	[code]	"a path outside the allow list is refused locally"
T3	A modified extension tries to write to Canvas through the backend	Integrity	<code>GET</code> only; <code>405</code> for other methods; no write scopes	[code]	"a write is refused even on an allowed path"
T4	Parameter tampering to widen Canvas's response (<code>all_dates</code> , <code>include[]=user</code>)	Data	Parameters and values validated; the rest refused	[code]	"an unknown query parameter is refused"
T5	Encoded <i>path traversal</i> (<code>..%2F</code>) to escape the pattern	Data	Decoding before comparison; <code>..</code> and <code>//</code> refused	[code]	"path traversal is refused"
T6	A malicious web page in the same browser calls the backend	Data	CORS with an origin list; unknown origin → <code>403</code> with no data	[code]	"a browser origin outside the list gets <code>403</code> "

#	Threat	Asset	Control	State	Test
T7	A request loop exhausts the Canvas quota and exposes the token to suspension	Token	Per-client rate limit, 429	[code]	"rate limit trips after the configured burst"
T8	The token appears in logs, errors or responses	Token	Logger without headers or query; errors without the Canvas body; describe() without the token	[code]	"health names the Canvas host and never the token", "log lines carry no query strings"
T9	A tampered pagination Link redirects the token to another domain	Token	The backend verifies each page stays on the configured Canvas host	[code]	—
T10	Network eavesdropping between backend and Canvas	Token, data	Mandatory HTTPS; the backend does not start with http:// outside localhost	[code]	—
T11	Impersonation: someone claims to be another student	Identity	Pilot: the backend infers identity from the token, not from the extension. Production: JWT signed by Canvas (LTI 1.3)	[code] / [pending]	"launch resolves the token owner"
T12	Intermediate caches keep academic data	Data	Cache-Control: no-store on every response	[code]	"responses carry no-store and nosniff"
T13	A compromised third-party package reads the token from the backend's memory	Token	Zero dependencies in the backend	[code]	—
T14	The AI service receives personal data	Identity, trust	Eleven-field context built in a single function; no name, email, ids, grades	[code]	—
T15	FARO becomes a surveillance tool (who studies how much)	Trust	Aggregate presence in the data model; no ranking or followers; view counts discarded	[code]	—
T16	A dropout-risk classification is exposed to the student	Trust	Friction states are internal; the interface uses supportive language; no risk score exists	[code]	—
T17	An instructor or admin token in the pilot widens the blast radius of a leak	Token, data	Policy: the pilot uses student-account tokens only	Policy	—

#	Threat	Asset	Control	State	Test
T18	The backend starts without a credential and serves empty data as if real	Trust	Refuses to start without <code>CANVAS_ACCESS_TOKEN</code> ; with an invalid token answers <code>canvas_token_rejected</code>	[code]	"a wrong token is reported as rejected"
T19	A student account cannot read analytics and FARO invents activity	Trust	<code>401 / 403</code> is passed to the client; FARO uses submission dates and says so on screen	[code]	"Canvas 401 on analytics is passed through"
T20	Theft of the <code>server/.env</code> file	Token	Scope limited to reading one student; one-click revocation in Canvas; the OAuth2 path removes the file	Policy / [pending]	—

6.3 What this model does not cover yet

- **Authentication of the extension to the backend in production.** Today the backend trusts the origin (CORS) and listens only on `127.0.0.1`. In an institutional deployment, the session issued after the LTI launch is what authorises each call. **[pending]**
- **Encryption at rest** of the future database. **[pending]**
- **Access auditing** by the institution: who queried what and when. The backend logs path and status per request; persistence and a pseudonymous id are missing. **[pending]**
- **External security review.** This document is a self-assessment; an institutional pilot must include a review by the university's security team.

7. Compliance: LFPDPPP and institutional policies

This chapter is technical guidance to prepare the institution's legal review. It does not replace the opinion of the university's legal department or data-protection officer.

7.1 The applicable framework in Mexico

The **Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)** was published in the Diario Oficial de la Federación on 20 March 2025 and entered into force on 21 March 2025. It replaces the 2010 law. The regulator for the private sector is no longer INAI, which was dissolved, but the **Secretaría Anticorrupción y Buen Gobierno**.

Three points of the new law that touch FARO directly:

1. **Definition of personal data.** Any information about an identified *or identifiable* person, "when their identity can be determined directly or indirectly through any information". A numeric Canvas id is therefore personal data even without a name: it is indirectly identifiable by the institution.
2. **Privacy notice.** It must identify sensitive data and distinguish the purposes that require express consent. When data is collected electronically, a simplified notice must be offered that refers to the full one.
3. **Consent.** It may be express or tacit; it is tacit when, the privacy notice having been made available, the data subject does not object. Sensitive data still requires express consent.

Note on the regulated party. A private university such as Tecmilenio is a private party and the LFPDPPP applies. A public university is governed by the Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. Both share the principles; this chapter's analysis holds for both, but the instrument and the authority change. **The institution must confirm which applies.**

7.2 The principles, one by one

Principle	What it requires	How FARO meets it	Mark
Lawfulness	Process data according to law and without deception	FARO only reads what the student can already see about themselves in Canvas; the request log is visible on screen	[code]
Consent	Have the data subject's will	Pilot: express, informed consent from each participant (template in 7.4). Production: privacy notice embedded in the LTI launch	Policy / [pending]

Principle	What it requires	How FARO meets it	Mark
Information	An accessible privacy notice	The <i>Profile</i> → <i>Canvas</i> screen and the Mentor footer show what is queried and what is sent; the formal notice is to be drafted with the institution	[code] / [pending]
Quality	Accurate, up-to-date data	Canvas is the source of truth; FARO does not copy the record, it reads it on each open	[code]
Purpose	Use data only for the stated purpose	Every Canvas path has a purpose written next to it (<code>server/src/allowlist.ts</code> , <code>purpose</code> field); there is no secondary use	[code]
Loyalty	No data obtained by deceptive means	No tracking beyond the 6 paths; no <i>dark patterns</i> to obtain the profile (everything is optional)	[code]
Proportionality	Only the data that is necessary	Chapter 5: view counts discarded, name and avatar from <code>users/self</code> discarded, mentor with 11 fields	[code]
Accountability	Ensure compliance and be able to demonstrate it	This book; the 20 automated tests; the checklist in chapter 9	[test]

7.3 Sensitive data

FARO **does not process sensitive data** as the law defines it (ethnic origin, health, beliefs, political opinions, sexual preference, genetic data). Two nuances the institution should consider:

- The *"life happened"* state (less time, overwhelmed, need a break) is self-declared, used only to choose an intervention, **never persisted and never leaves the browser**. It is not health data, but the privacy notice should mention it to leave no doubt.
- The **destination sentence** is free text. If a student wrote sensitive data there, FARO would store it locally and include it in the mentor context. Mitigation: an on-screen warning **[pending]** and a policy of not storing conversations.

7.4 Consent for the pilot

For a pilot with five to thirty students, written express consent is recommended. Minimum elements it must contain:

1. Who the controller is (the institution) and who the processor is (the FARO team).
2. What data is read from Canvas (the six paths, in plain language).
3. What data FARO creates (purpose, sessions, preferences, optional profile).
4. That FARO modifies nothing in Canvas.
5. That the AI mentor, once enabled, will receive a context with no name or email.
6. How long data is kept and how to request deletion (*Progress* → *Reset*, and token revocation).
7. That participation is voluntary and withdrawing does not affect grades.

8. How to exercise ARCO rights before the institution.

7.5 ARCO rights

Right	How it is exercised in FARO today	In production
Access	<i>Profile</i> shows everything FARO stores about the student; <i>Profile</i> → <i>Canvas</i> shows everything it reads	Export from the backend
Rectification	Academic data is corrected in Canvas (source of truth); FARO data, in <i>Profile</i>	Same
Cancellation	<i>Progress</i> → <i>Reset</i> wipes local state; uninstalling the extension removes everything	Database deletion on request
Opposition	Do not install the extension; withdraw pilot consent	Disable the LTI tool for that student

7.6 Transfers

- **To Canvas (Instructure):** FARO transfers nothing to Canvas; it only reads. The institution's relationship with Instructure is already covered by its existing contract.
- **To the AI model provider [pending]:** this will be a transfer of the 11 context fields plus the student's message. It must appear in the privacy notice, and the contract with the provider must forbid using that data for training. Until then, the mentor is local and there is no transfer.
- **Between the student's devices [pending]:** once the backend with a database exists, FARO's state will sync through it. That is processing by the processor, not a transfer to a third party.

7.7 Institutional Canvas policies

Regardless of the law, the institution controls in Canvas:

- Whether students can generate access tokens and with what expiry.
- Which scopes it grants to FARO's developer key (recommendation: exactly those in `requiredScopes`).
- Whether the LTI tool is installed at account, sub-account or course level.
- Whether students can view their own analytics (this determines whether FARO uses the activity endpoint or the submission-based fallback).

None of these decisions requires changes in FARO; the backend respects what Canvas allows and reports what it does not.

8. Frequently asked questions for the university

The questions an IT department, a data-protection officer or an academic director asks before connecting a tool to Canvas, with short answers and the reference to check them.

Access and scope

Can FARO modify grades, submissions or due dates? No. Every call to Canvas is a read (`GET`). The backend refuses any other method before contacting Canvas, and the LTI scopes that would allow writing to the gradebook (`score` , `lineitem`) are explicitly refused. Check `server/src/allowlist.ts` and the developer key screen in Canvas.

What exactly does FARO see in Canvas? Six paths: the course, its modules with items, the assignments with the student's own submission, the student's own activity (timestamps), the student's list of active courses, and their numeric id. Nothing else. The list is a thirty-line file: `server/src/allowlist.ts` .

Does FARO see other students' data? No. It does not call the enrollments endpoint or the course users endpoint. In production, the "how many are studying" count will come from LTI's NRPS service in aggregate mode, with no names.

Can FARO see the contents of a submission or an exam? No. It reads the submission's date, state and score, not its contents. The fields that are read are declared in `src/data/canvas/raw.ts` ; whatever is not there is ignored.

Can a student use FARO to see someone else's course? No. In the pilot, the backend infers identity from the token, not from what the extension claims. In production, identity is signed by Canvas (LTI 1.3) and FARO accepts no other.

Credentials

Where is the Canvas token? In `server/.env` , on the machine running the backend, and in that process's memory. Nowhere else. The extension has no field for a token and no network permission for `instructure.com` .

Is the token in the source code or on GitHub? No. `.env` is in `.gitignore` . The repository contains `server/.env.example` with the field empty.

What if the token leaks? Revoke it in Canvas with one click, generate another, restart the backend. The blast radius is reading what that student sees about themselves. That is why the pilot uses only student-account tokens, never instructor or administrator ones. Chapter 4.

Does every student need to generate a token? In the pilot, yes, one per participant (or a test-account token). That is the practice Instructure indicates for testing. For a real deployment, FARO uses OAuth2 with a developer key and Canvas issues tokens automatically, valid for one hour.

Artificial intelligence

What does the AI model receive? Eleven fields: course name, progress percentage, next activity and its duration, declared goal, destination sentence, available minutes, momentum, friction state, style and language. It is in `src/lib/mentorContext.ts`. It does not receive name, email, ids, grades or history.

Can the model do the student's homework? No. There is a filter for "do my exam"-type requests that answers with a refusal and offers to break the task down, explain the concept or review the reasoning. The filter is the same across the five communication styles.

Does data go to an AI provider today? No. The prototype's mentor is local and deterministic. The model call will be enabled from the backend once there is a contract with the provider and an updated privacy notice.

Is the data used to train models? No, and the contract with the provider must explicitly forbid it before the remote mentor is enabled.

Surveillance and wellbeing

Does FARO compute a student's "dropout risk"? It computes internal friction states (five labels based on days without activity and pending work) to decide which intervention to offer. It **never** shows a risk score to the student and shares it with nobody. The interface says "your route needs attention", not "risk: 87%".

Can an instructor see who studies how much? That screen does not exist, and Community's data model only admits aggregate counts, not people. If the institution wanted pilot metrics, the proposal is to deliver them aggregated and with pseudonymous ids.

Does FARO send notifications or reminders? No. There are no push notifications, emails or reminders. FARO waits for the student to open the panel. The "safe harbour" (planned pause) explicitly has no countdown.

Is FARO a video game? No. It uses mechanics (route, checkpoints, achievements) without lives, levels or leaderboards. The five achievements and five recognitions require having persisted; none is earned by clicking.

Operations

Where does the backend run? In the pilot, on a team or institution machine, listening on `127.0.0.1`. In production, on institutional infrastructure or an approved provider, behind TLS, with the LTI launch as authentication.

What dependencies does it have? The backend, none. Only Node.js. The extension, two: `react` and `react-dom`.

How do I know the guarantees in this book are true? Run `npm test` inside `server/`: twenty automated checks against a fake Canvas with real pagination. And open *Profile* → *Canvas* in FARO: the live request log shows every call.

What is missing for production? LTI 1.3 registration by the institution, OAuth2 per student, a database with encryption at rest, a remote mentor with a data contract, a formal privacy notice and an external security review. Chapter 9 has the full list.

Does FARO replace Canvas? No. Canvas remains the academic source of truth. FARO reads from Canvas and adds a companionship layer. *We don't replace Canvas: we make it adaptive.*

9. Pre-pilot checklist

Tick every item before connecting FARO to a Canvas with real students. Items marked ⚙ are verified by the technical team; items marked 🏛 by the institution.

A. Before starting the backend

- ☐ ⚙ `server/.env` exists, is **not** in git (`git status` does not list it) and `CANVAS_API_URL` begins with `https://`.
- ☐ ⚙ The token belongs to a pilot **student account**, not an instructor or administrator.
- ☐ ⚙ The token has an **expiry date** (recommended: 30 days).
- ☐ ⚙ `FARO_HOST=127.0.0.1` unless TLS sits in front.
- ☐ ⚙ `FARO_ALLOWED_ORIGINS` contains only the real origins (the loaded extension's id, the dev server if applicable).
- ☐ ⚙ `npm test` in `server/` passes all 20 checks.
- ☐ ⚙ `curl http://127.0.0.1:3000/health` answers with the Canvas host and does **not** contain the token.

B. Before loading the extension

- ☐ ⚙ It was built with `VITE_CANVAS_MODE=http` and `VITE_FARO_API_URL` pointing at the backend.
- ☐ ⚙ `public/manifest.json` does **not** contain `instructure.com` in `host_permissions`.
- ☐ ⚙ In *Profile* → *Canvas* the badge says *Live* and the log shows status `200` on all four paths.
- ☐ ⚙ If the note "your account cannot read analytics" appears, it was decided with the institution whether to enable the permission or accept the submission-based fallback.

C. With the institution

- ☐ 🏛 Which law applies has been confirmed (LFPDPPP for a private institution; Ley General for a public one).
- ☐ 🏛 A privacy notice exists (simplified + full), reviewed by legal, mentioning: the six Canvas paths, the data FARO creates, the "life happened" state, the future remote mentor, and retention.
- ☐ 🏛 Every participant signed the pilot consent (chapter 7.4).
- ☐ 🏛 It was agreed how ARCO rights are exercised during the pilot and who responds.
- ☐ 🏛 It was agreed which pilot metrics are delivered and in what form (aggregated, with pseudonymous ids).
- ☐ 🏛 The institution's security team reviewed this book and the `server/` code.

- ☐ ⚙ The pilot end date and the procedure for revoking tokens and deleting data at the end were defined.

D. When the pilot ends

- ☐ ⚙ All pilot tokens revoked in Canvas.
- ☐ ⚙ `server/.env` deleted from the backend machine.
- ☐ ⚙ Participants told how to delete local state (*Progress* → *Reset* or *uninstall*).
- ☐ ⚙ Metrics delivered; raw data destroyed or handed to the institution as agreed.

E. To move to production (outside the pilot's scope)

- ☐ Registration of FARO as an account-level LTI 1.3 tool in Canvas, with exactly the scopes in `requiredScopes` and without `score` or `lineitem`.
- ☐ Implementation of `/lti/login`, `/lti/launch` and `/.well-known/jwks.json` per the specification in `server/src/routes/lti.ts`.
- ☐ OAuth2 per student; removal of the shared token.
- ☐ PostgreSQL with encryption at rest; automatic retention policy.
- ☐ Remote mentor behind the backend, with a data contract that forbids training.
- ☐ External security review.